

Understanding `dict()` , `exclude_unset` , and `exclude` in Pydantic (FastAPI)

Introduction

When working with FastAPI and Pydantic models, one of the most common patterns used in create and update APIs is:

```
req.dict()
```

or

```
req.dict(exclude_unset=True)
```

At first, this can feel confusing because fields appear “automatically” in the final dictionary even when they are not manually written in the code.

This document explains:

- What `dict()` actually does
- What `exclude_unset=True` means
- What `exclude={...}` means
- When to use them
- When NOT to use them
- Their alternatives
- Real-world backend design patterns

1. What is `req.dict()` ?

Suppose we have a Pydantic model:

```
class CreateCategoryRequest(BaseModel):  
    name: str
```

```
description: Optional[str] = None
escalation_levels: int
```

And the user sends this payload:

```
{
  "name": "Support",
  "escalation_levels": 2
}
```

FastAPI converts the payload into a Python object:

```
req
```

Now if we do:

```
req.dict()
```

It becomes:

```
{
  "name": "Support",
  "description": None,
  "escalation_levels": 2
}
```

This means:

- `dict()` converts the Pydantic model into a normal Python dictionary.
-

2. What is `exclude_unset=True` ?

By default, `dict()` includes all fields, even if the user did not send them.

Example:

```
req.dict()
```

Output:

```
{
  "name": "Support",
  "description": None,
  "escalation_levels": 2
}
```

Notice:

- `description` is included even though the user never sent it.

Now:

```
req.dict(exclude_unset=True)
```

Output:

```
{
  "name": "Support",
  "escalation_levels": 2
}
```

Now only the fields actually sent by the user are included.

3. Why Use `exclude_unset=True` ?

This is extremely useful in:

- Create APIs
- Update APIs
- Patch APIs
- Dynamic payload handling

Benefits:

- Cleaner payloads
- Smaller database documents
- Avoids unnecessary `None` values

- Makes updates safer
-

4. What Does `**req.dict()` Mean?

Example:

```
data = {  
    **req.dict(exclude_unset=True),  
    "org_id": "123"  
}
```

Suppose:

```
req.dict(exclude_unset=True)
```

returns:

```
{  
    "name": "Support"  
}
```

Then final result becomes:

```
{  
    "name": "Support",  
    "org_id": "123"  
}
```

The `**` operator spreads all key-value pairs into another dictionary.

5. What is `exclude={...}` ?

This is where many developers get confused.

Example:

```
req.dict(  
    exclude_unset=True,  
    exclude={"assignment_status"}  
)
```

Meaning:

Take all payload fields
EXCEPT assignment_status

6. Why Do We Exclude Fields?

We exclude fields when:

- We want to manually process them
 - We want custom logic
 - We want transformation
 - We want renaming
 - We want validation
 - We want default handling
-

7. Real Example

Payload:

```
{  
    "title": "Math Test",  
    "assignment_status": "DRAFT"  
}
```

Suppose we want:

```
'status': req.assignment_status or "DRAFT"
```

Notice:

- payload field = `assignment_status`
- DB field = `status`

We are:

- renaming the field
- applying logic

So we do:

```
assignment_data = {  
    **req.dict(  
        exclude={"assignment_status"}  
    ),  
  
    "status": req.assignment_status or "DRAFT"  
}
```

Without `exclude` , both fields may exist:

BAD:

```
{  
    "assignment_status": "DRAFT",  
    "status": "DRAFT"  
}
```

Correct:

```
{  
    "status": "DRAFT"  
}
```

8. Golden Rule

DO NOT exclude:

Fields that can go directly from payload to database.

Example:

```
title
description
name
```

Exclude:

Fields that require processing.

Example:

- Renaming
 - Default values
 - Validation
 - Conversion
 - Transformation
 - Business logic
-

9. Common Real-World Examples

Example 1 — Rename Field

Payload:

```
{
  "assignment_status": "DRAFT"
}
```

DB:

```
"status": req.assignment_status
```

Use exclude? YES

Example 2 — Type Conversion

Payload:

```
{  
  "set_time_minutes": "15"  
}
```

DB:

```
"set_time_minutes": int(req.set_time_minutes)
```

Use exclude? YES

Example 3 — Default Fallback

Payload:

```
{  
  "proctoring_configurations": null  
}
```

DB:

```
"proctoring_configurations": (  
    req.proctoring_configurations.dict()  
    if req.proctoring_configurations  
    else DEFAULT_PROCTORING_CONFIG  
)
```

Use exclude? YES

Example 4 — Direct Save

Payload:


```
{  
  "title": "Math Test"  
}
```

DB:

```
"title": req.title
```

Use exclude? NO

10. When NOT to Use `dict(exclude_unset=True)`

Sometimes explicit mapping is better.

Example:

```
{  
  "name": req.name,  
  "role_id": req.role_id  
}
```

Useful when:

- Complex enterprise systems
 - Very strict schemas
 - Security-sensitive APIs
 - Heavy nested objects
 - Multi-team backend systems
-

11. Hybrid Approach (Best Industry Pattern)

Most senior backend engineers use a hybrid approach.

Example:

```
assignment_data = {  
    **req.dict(  
        exclude_unset=True,  
        exclude={"assignment_status"}  
    ),  
  
    "status": req.assignment_status or "DRAFT",  
  
    "org_id": user["org_id"]  
}
```

This gives:

- automation
- flexibility
- readability
- control

all together.

12. Simple Mental Model

req.dict()

Means:

Convert payload model into dictionary

exclude_unset=True

Means:

Only include fields sent by the user

exclude={...}

Means:

Do not automatically include these fields
I will manually handle them

13. Final Summary

Situation	What to Do
Direct payload DB	Do NOT exclude
Payload + processing	Exclude
Payload + transformation	Exclude
Payload + rename	Exclude
Payload + business logic	Exclude
Internal backend fields	Add manually

14. Final Recommendation

Small/Medium APIs

Use:

```
{  
  **req.dict(exclude_unset=True)  
}
```

Complex APIs

Use Hybrid:

```
{  
  **req.dict(  
    exclude_unset=True,  
    exclude= {...}  
  ),  
  
  # custom processed fields  
}
```

15. Most Important Line to Remember

exclude means:
"Do not automatically include this field,
because I want to manually process it."